# 2026-05-25 - Custom PCB Board Definition in Zephyr

## Goal

Learn how to define a custom PCB as a named Zephyr board target so that your hardware is fully described by Device Tree and Kconfig rather than relying on a stock evaluation kit.

## What you will learn

- The file structure required for a custom board
- How to write a board-level `.dts` and `_defconfig`
- How to register GPIO, UART, I2C, and SPI peripherals for your PCB
- How to build and flash targeting your custom board name

## Overview

When you design your own PCB around a chip like the nRF52840, you create a board definition that tells Zephyr exactly which peripherals are wired up and how. This replaces the DK-specific assumptions in `nrf52840dk/nrf52840`.

### Required file structure

```
boards/
└── arm/
    └── my_custom_board/
        ├── Kconfig.board          ← board name declaration
        ├── Kconfig.defconfig      ← default Kconfig symbols
        ├── board.cmake            ← flash runner (JLink, OpenOCD, etc.)
        ├── my_custom_board.dts    ← hardware description
        ├── my_custom_board.yaml   ← board metadata
        └── my_custom_board_defconfig  ← minimal kernel config
```

### my_custom_board.dts (skeleton)

```
/dts-v1/;
#include <nordic/nrf52840_qiaa.dtsi>

/ {
    model = "My Custom Board";
    compatible = "my-company,my-custom-board";
```

```
    chosen {
        zephyr,console = &uart0;
        zephyr,shell-uart = &uart0;
        zephyr,sram = &sram0;
        zephyr,flash = &flash0;
    };

    leds {
        compatible = "gpio-leds";
        led0: led_0 {
            gpios = <&gpio0 13 GPIO_ACTIVE_LOW>;
            label = "Status LED";
        };
    };

    aliases {
        led0 = &led0;
    };
};

&uart0 {
    status = "okay";
    compatible = "nordic,nrf-uarte";
    current-speed = <115200>;
    pinctrl-0 = <&uart0_default>;
    pinctrl-names = "default";
};

&i2c0 {
    status = "okay";
    pinctrl-0 = <&i2c0_default>;
    pinctrl-names = "default";
    clock-frequency = <I2C_BITRATE_FAST>;
};
```

### Kconfig.board

```
config BOARD_MY_CUSTOM_BOARD
    bool "My Custom Board"
    select SOC_NRF52840_QIAA
```

### board.cmake

```
board_runner_args(jlink "--device=nRF52840_xxAA" "--speed=4000")
include(${ZEPHYR_BASE}/boards/common/jlink.board.cmake)
```

### my_custom_board_defconfig

```
CONFIG_SOC_SERIES_NRF52X=y
CONFIG_SOC_NRF52840_QIAA=y
CONFIG_BOARD_MY_CUSTOM_BOARD=y
```

```
CONFIG_SYS_CLOCK_HW_CYCLES_PER_SEC=64000000
CONFIG_GPIO=y
CONFIG_SERIAL=y
```

### Building for your custom board

Place the `boards/` directory either: - Inside your application directory, or - In a separate module registered via `west.yml`

Then build with:

```
west build -b my_custom_board
```

### Pin control (nRF52840 specific)

nRF52840 uses `pinctrl` to assign peripheral functions to physical pins. Add a `pinctrl` block to your `.dts`:

```
&pinctrl {
    uart0_default: uart0_default {
        group1 {
            psels = <NRF_PSEL(UART_TX, 0, 6)>,
                    <NRF_PSEL(UART_RX, 0, 8)>;
        };
    };
    i2c0_default: i2c0_default {
        group1 {
            psels = <NRF_PSEL(TWIM_SDA, 0, 26)>,
                    <NRF_PSEL(TWIM_SCL, 0, 27)>;
        };
    };
};
```

Replace pin numbers with those from your PCB schematic.

## Why this matters

Every production product needs a custom board definition. Keeping hardware description in DTS and Kconfig means your application code stays portable and board-agnostic.

## Practice tasks

1. Copy the `nrf52840dk` board directory from the Zephyr tree and rename it to `my_custom_board`.
2. Change the LED GPIO pin to match a pin on your schematic.

3. Build `src/2026-05-25_Custom_PCB_Board_Definition` targeting `my_custom_board`.

## Example folder

See `src/2026-05-25_Custom_PCB_Board_Definition` for the day's code and a sample `boards/` skeleton.

## Next topic

Tomorrow we enable USB CDC-ACM so the board can communicate with a PC over USB.